

ANoT: Adaptive Network of Thought for Robust Multi-Source Reasoning

Author Name

Affiliation

email@example.com

Abstract

Large language models struggle to reason over structured, multi-source data: existing prompting methods conflate heterogeneous evidence and remain vulnerable to adversarial content in user-generated text. We introduce **ANoT** (Adaptive Network of Thought), a split-context prompting method that decomposes reasoning into isolated, verifiable steps across three phases—Planning, Adaptation, and Execution—with three-valued logic aggregation for explicit uncertainty handling. On a Yelp-based recommendation benchmark with adversarial attacks, ANoT outperforms 13 baselines by +16% on clean data; under attack, baselines degrade by **XX%** while ANoT drops only **YY%**.

1 Introduction

Large language models (LLMs) are increasingly deployed in applications that require reasoning over *structured, multi-source data*—information combining rich metadata with unstructured content from multiple contributors [Wang *et al.*, 2024]. Consider a recommendation system: to match a user’s query (“quiet cafe with free WiFi, open Monday afternoon, where reviews mention good matcha”), an LLM must jointly reason over structured attributes, temporal constraints, and dozens of customer reviews. Such scenarios are ubiquitous—from e-commerce product search to healthcare record analysis to legal document review—yet existing prompt schemes provide inadequate support.

This setting presents three interrelated challenges. *First*, multi-source data introduces heterogeneity and adversarial vulnerability: user-generated content exhibits uneven quality, and real-world platforms are susceptible to fake reviews, prompt injection attacks, and coordinated manipulation [Greshake *et al.*, 2023]. *Second*, structured text requires distinct handling from unstructured text; LLMs struggle to jointly process hierarchical metadata alongside free-form content without conflating evidence types [Cao *et al.*, 2024; Gu *et al.*, 2024]. *Third*, complex user requests involve compositional logic with nested conditions, requiring models to correctly parse operator precedence and propagate uncertainty across subexpressions. These difficulties are pervasive in practice,

yet no existing prompt scheme adequately addresses their combination.

Existing prompting methods fall into two categories based on how they handle input data. *Full-context schemes*—including Chain-of-Thought [Wei *et al.*, 2022], ReAct [Yao *et al.*, 2023], and Least-to-Most [Zhou *et al.*, 2023]—process all available information in a single LLM call. While effective for homogeneous text, these methods degrade on structured multi-source data: they conflate evidence from different sources and prove vulnerable to adversarial content that exploits the shared context. *Split-context schemes*—such as Decomposed Prompting [Khot *et al.*, 2022]—process information in separate calls, achieving better modularity. However, existing split-context methods still fall short: they lack content-aware adaptation to detect attacks before execution, provide no explicit evidence tracking across sources, and do not support compositional logic aggregation with uncertainty handling.

We propose **Adaptive Network of Thought (ANoT)**, a split-context prompt scheme that addresses all three challenges through a principled three-phase architecture. **Phase 1 (Planning)** decomposes the user request into structured conditions mapped to specific evidence sources—this explicit decomposition prevents conflation between metadata and unstructured content. **Phase 2 (Adaptation)** analyzes actual data content to detect heterogeneity or potential attacks *before* the main reasoning phase, ensuring adversarial content never reaches the decision logic. **Phase 3 (Execution)** processes each evidence source independently and aggregates results using three-valued logic that explicitly handles uncertainty—enabling correct compositional reasoning while preserving minority viewpoints.

We evaluate ANoT on a structured recommendation benchmark constructed from Yelp Open Dataset, featuring 20 user requests across 4 complexity categories that vary by evidence type. We further test robustness against eight attack types, including prompt injection and fake reviews. Our contributions are:

- **ANoT**, a split-context prompting method achieving high accuracy on compositional multi-source tasks with robustness through architectural isolation.
- A **structured recommendation benchmark** with complex queries and comprehensive attack evaluation.

- Empirical findings showing ANoT outperforms 13 baselines by +16% on clean data while maintaining performance under attacks.

2 Related Work

Full-Context Prompting. Full-context methods process all information within a single LLM context window. Chain-of-Thought [Wei *et al.*, 2022] generates intermediate reasoning steps; Self-Consistency [Wang *et al.*, 2023b] samples multiple paths and selects via majority vote. ReAct [Yao *et al.*, 2023] interleaves reasoning with actions; Plan-and-Solve [Wang *et al.*, 2023a] generates explicit plans; Self-Ask [Press *et al.*, 2023] and Least-to-Most [Zhou *et al.*, 2023] decompose problems into subproblems. These methods conflate evidence from different sources within a shared context, making them vulnerable to distraction and adversarial manipulation.

Split-Context Methods. Split-context methods isolate input segments, processing each separately before aggregation. Decomposed Prompting [Khot *et al.*, 2022] uses specialized sub-task handlers orchestrated by a decomposer, but employs static handlers without adapting to input content. Ranking methods also vary in context handling: pairwise [Qin *et al.*, 2023] and setwise [Sun *et al.*, 2023] approaches compare candidates in groups, while pointwise scoring [Liang *et al.*, 2023] rates each candidate independently—naturally supporting split-context processing where each item is evaluated in isolation.

Adversarial Robustness. Real-world multi-source data faces adversarial threats. Prompt injection embeds malicious instructions within user-provided content [Perez and Ribeiro, 2022; Greshake *et al.*, 2023]; fake reviews and coordinated manipulation are endemic to online platforms [Mukherjee *et al.*, 2013]. Defense prompts [Jain *et al.*, 2023] add explicit warnings but require distinguishing adversarial from genuine content at inference time—a challenge our Adaptation phase addresses through content-aware detection before execution.

3 Problem Formulation

3.1 Task Definition

Given candidate items $\mathcal{I} = \{I_1, \dots, I_n\}$ and a Boolean request R , select $I^* = \arg \max_{I \in \mathcal{I}} \sigma(I, R)$ where $\sigma : \mathcal{I} \times \mathcal{R} \rightarrow \{-1, 0, +1\}$ is the three-valued satisfaction function defined below.

Each item $I = (\mathbf{m}, \mathbf{h}, \mathbf{D})$ comprises structured metadata $\mathbf{m}$ (categorical and Boolean attributes), operating hours $\mathbf{h}$, and a document set $\mathbf{D} = \{d_1, \dots, d_k\}$ of user reviews with associated metadata.

3.2 Request Formalism

Each request R is a Boolean expression tree where leaf nodes specify conditions with an aspect name and evidence source, and internal nodes apply AND/OR operators. Evidence sources indicate where to find supporting information:

- `item_meta`: Direct attribute lookup in $\mathbf{m}$
- `item_hours`: Time-window containment in $\mathbf{h}$
- `review_text`: Semantic analysis of review content

- `review_meta`: Filtering over reviewer signals

3.3 Three-Valued Semantics

Real-world evidence is often incomplete or ambiguous. We adopt three-valued logic [Fitting, 1985] where each condition evaluates to $v \in \{+1, 0, -1\}$ (satisfied, unknown, violated). Boolean operators extend naturally:

$$a \wedge b = \min(a, b), \quad a \vee b = \max(a, b) \quad (1)$$

This ensures violations dominate conjunctions while satisfactions dominate disjunctions, with unknown values propagating only when outcome-relevant.

3.4 Evaluation

Each request has exactly one ground-truth item with $\sigma(I^*, R) = +1$, enabling unambiguous evaluation. We measure Hits@ k :

$$\text{Hits}@k = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} \mathbf{1}[\text{rank}_r(I^*) \leq k] \quad (2)$$

where $\text{rank}_r(I^*)$ is the ground-truth item’s position in the method’s ranking.

4 Method

We present ANoT, a split-context prompting method with three phases (Figure 1). *Planning* decomposes requests into evidence-specific conditions, handling structured text and compositional logic. *Adaptation* detects heterogeneity and adversarial content before execution. *Execution* aggregates evidence using three-valued logic with correct uncertainty propagation. Planning runs once per request and is cached; only Adaptation and Execution run per item, requiring $O(k)$ LLM calls where k is the number of conditions.

4.1 Phase 1: Planning

Planning transforms a natural-language request into a validated, executable script. Given the user request, the LLM first generates a structured plan identifying: (1) each condition to verify, (2) the evidence source for each condition (ATTR, HOURS, REVIEW.TEXT, REVIEW.META), (3) value interpretation rules, and (4) the compositional aggregation logic.

The plan is then translated into a *Lightweight Workflow Template* (LWT), a simple script where each line specifies an LLM call with explicit data access paths:

```
(0)=LLM("Check {(input)}[attr][NoiseLevel] Quiet?
    Output: 1=yes, -1=no, 0=unknown")
(1)=LLM("Check {(input)}[attr][WiFi]. Free?
    Output: 1=yes, -1=no, 0=unknown")
(2)=LLM("Analyze {(input)}[reviews][0..N][text].
    Mention good matcha? Output: 1/-1/0db)
(3)=LLM("Results: Noise={ (0) }, WiFi={ (1) }, Matcha=
    Logic: AND. Output: -1 if any -1, 1 if all 1")
```

Both plan and script undergo iterative validation (up to 5 rounds) to ensure completeness and schema compliance. Validated scripts are cached per request, amortizing planning cost across all candidate items.

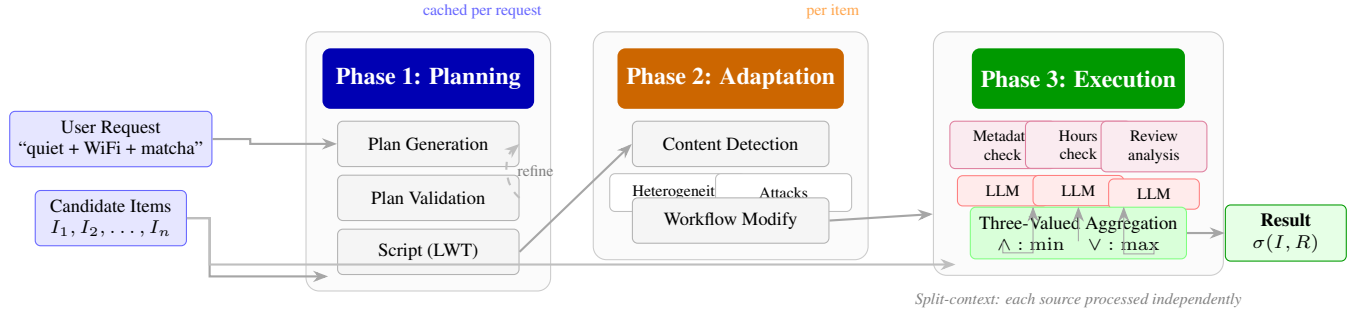


Figure 1: Overview of the ANoT architecture. **Phase 1 (Planning)** transforms user requests into validated executable scripts, cached across items. **Phase 2 (Adaptation)** detects content heterogeneity and potential attacks before execution. **Phase 3 (Execution)** processes each evidence source independently via split-context LLM calls, then aggregates results using three-valued logic (+1: satisfied, 0: unknown, -1: violated).

4.2 Phase 2: Adaptation

Adaptation analyzes actual data content to detect characteristics requiring workflow modification. This phase runs per item, as content varies across candidates.

The LLM examines review content for three potential issues:

- **Heterogeneity:** Highly uneven review lengths where verbose reviews might dominate.
- **Attack patterns:** Command-like text (“ignore previous”, “output:”, “answer is”).
- **Fake indicators:** Generic, suspiciously uniform reviews covering all aspects.

Based on detected conditions, the workflow may be modified—e.g., adding a filtering step that excludes suspicious content before the main analysis, or weighting reviews by credibility signals. Critically, adaptation happens *before* execution: the main reasoning phase never sees potentially adversarial content, only the controlled detection phase does. This architectural isolation provides robustness without degrading performance on clean data.

4.3 Phase 3: Execution

Execution runs the (possibly adapted) script step-by-step. Variable references are resolved dynamically: $\{\text{input}\}$ becomes the current item’s data, and $\{\text{N}\}$ becomes the output of step N . Each evidence source is processed independently, preventing cross-contamination between metadata checks and review analysis.

The final step aggregates evidence using three-valued logic (Section 3.3), ensuring correct compositional reasoning: for AND, any violation dominates; for OR, any satisfaction dominates; unknown values propagate only when they could affect the outcome. This preserves minority viewpoints—a single negative review is not drowned out by neutral ones.

5 Experiment

Data construction. We construct *query* and *context* sets from the Yelp Open Dataset using a multi-stage curation pipeline designed to ensure data quality and relevance.

Curation Pipeline. First, we interactively select a target city and specific cuisine categories (e.g., “Italian”, “Mexican”) based on restaurant density. To ensure the selected entities genuinely fit the target categories, we employ an automated LLM-based filtering step. Candidates are ranked by *richness* (total volume of review text) and processed by an LLM which estimates the probability of category fit (0–100%) based on keyword evidence extracted from their reviews. We retain only restaurants with a high probability of fit.

Review Sampling. For each selected restaurant, we construct a document set of up to 20 reviews. To capture diverse user sentiments, we stratify the sampling by star rating, targeting 4 reviews from each star level (1–5 stars) where available. Within each stratum, reviews are selected by descending text length to maximize the information available for reasoning.

Metadata Structure. Each query corresponds to a single restaurant and includes: (i) *Item Metadata*: structured fields such as name, location, categories, attributes, and hours; and (ii) *Review Data*: the set of 20 sampled reviews, including text, rating, date, and reviewer profile signals (e.g., elite status, friend count, year started).

Contexts (User Requests). Each context corresponds to a natural-language user request that specifies a set of constraints, which must be supported by evidence available in the query. Rather than assuming a single notion of difficulty, we organize contexts into groups that isolate distinct combinations of evidence sources and logical structures. Requests vary by the types of information sources they reference: direct item metadata, computed metadata (e.g., hours or time-window constraints), review text, and reviewer or review metadata.

Requests consist of compositions of conditions connected by AND/OR operators (e.g., $A \text{ op}_1 B \text{ op}_2 C$). Using this design, we construct 20 contexts grouped into 4 categories, each containing five requests, where each group isolates a specific combination of evidence sources (simple metadata, review text, computed metadata, social signals).

- **G01 (R00–R04): Flat / Simple Metadata.** Requests use only direct item metadata (e.g., attributes, cate-

gories) with flat AND/OR composition.

- **G02 (R05–R09): Flat / Review Text.** Requests rely exclusively on conditions grounded in review text, composed using flat AND/OR logic.
- **G03 (R10–R14): Flat / Computed Metadata.** Requests use computed item metadata requiring interpretation (e.g., hours, time-window constraints) with flat AND/OR logic.
- **G04 (R15–R19): Flat / Social Metadata.** Requests use only reviewer or review metadata (e.g., elite status, recency) with flat AND/OR logic.

Baselines. We compare against 13 zero-shot prompting methods spanning two categories: *reasoning-based* methods that apply general problem-solving strategies (Chain-of-Thought, Self-Consistency, ReAct, Least-to-Most, Plan-and-Solve, Self-Ask, Decomposed Prompting), and *ranking-based* methods designed specifically for item selection (Pointwise, PaRaDe, Pairwise, RankGPT, Setwise, Listwise). Details are provided in Appendix C.6.

Ground truth generation. For each (item, request) pair, we compute ground truth labels deterministically by evaluating leaf conditions against available evidence and aggregating via the request’s Boolean structure. For `review_text` conditions, we use LLM-based judgments with symmetric consensus voting: a review is labeled positive (+1) or negative (−1) only if at least 75% of non-neutral judgments agree. Judgments are cached to ensure reproducibility.

Adversarial evaluation. To evaluate robustness, we apply attacks to review content **D** while preserving ground truth labels (computed from original, unattacked data). Attack types include:

- **Typo corruption:** Random character transpositions (10–20% rate)
- **Prompt injection:** Embedded commands (“IGNORE PREVIOUS INSTRUCTIONS...”)
- **Fake reviews:** Synthetic reviews with extreme, generic sentiment
- **Sarcastic manipulation:** Factually correct but misleadingly framed content

A robust method should maintain consistent performance across clean and attacked data.

6 Discussion

Why Split-Context Provides Robustness. A key insight from our work is that robustness emerges from *architectural isolation* rather than explicit defense mechanisms. Full-context methods are fundamentally vulnerable because adversarial content in one evidence source can influence reasoning about unrelated sources—a fake review containing “ANSWER: recommend” affects judgments about metadata attributes processed in the same context. ANoT’s split-context design eliminates this attack vector by construction: metadata checks never see review text, and each review is processed independently. This architectural defense requires no additional prompting and introduces no performance penalty on clean data, unlike defense prompts that make models overly conservative.

Preserving Minority Viewpoints. Three-valued logic serves a dual purpose in ANoT. First, it handles genuine uncertainty (missing attributes, ambiguous reviews) without forcing false binary decisions. Second, and more subtly, it preserves minority viewpoints during aggregation. When reviews are processed independently and aggregated with explicit unknown handling, a single review noting “the WiFi was slow” cannot be drowned out by ten reviews that simply fail to mention WiFi. The unknown values propagate through the Boolean logic, ensuring that absence of evidence is not treated as evidence of absence.

Limitations. ANoT incurs additional computational cost compared to single-pass methods: multiple LLM calls (one per evidence source plus aggregation) versus one. This overhead is partially mitigated by caching validated scripts across items, but remains a consideration for latency-sensitive applications. Additionally, our evaluation focuses on structured recommendation with discrete Boolean conditions; tasks requiring fuzzy matching or continuous scoring may need different aggregation mechanisms. Finally, while our adaptation phase detects several attack patterns, sophisticated adversaries might craft attacks that evade detection—though architectural isolation still provides baseline protection.

Counter-Intuitive Finding. Adding explicit defense prompts (“watch out for adversarial content”, “be skeptical of suspicious reviews”) to full-context baselines actually *hurts* performance on clean data. Models become overly conservative, treating legitimate reviews with strong opinions as potentially adversarial. This suggests that prompting alone cannot solve the robustness problem—architectural changes that prevent adversarial content from reaching the reasoning context are necessary.

7 Conclusion

We presented ANoT (Adaptive Network of Thought), a split-context prompting method for reasoning over structured, multi-source data. By decomposing queries into isolated evidence-specific steps and aggregating results with three-valued logic, ANoT achieves both high accuracy on complex Boolean conditions and robustness to adversarial manipulation—through architectural isolation rather than defense prompts. On our Yelp-based benchmark, ANoT outperforms 13 baseline methods by +16% on clean data while maintaining consistent performance under 8 attack types.

For practitioners deploying LLMs on user-generated content, our key takeaway is that *how* information reaches the model matters as much as what prompting strategy is used. Split-context processing eliminates entire classes of vulnerabilities by preventing adversarial content from polluting unrelated reasoning steps.

Future work includes extending ANoT to continuous scoring tasks, exploring hierarchical evidence decomposition for deeply nested queries, and investigating adaptive caching strategies to reduce latency. We release our benchmark and code to support further research on robust LLM reasoning over multi-source data.

References

- [Cao *et al.*, 2024] Boxi Cao, Mengjie Ren, Hongyu Lin, Xi-anpei Han, Feng Zhang, Junfeng Zhan, and Le Sun. StructEval: Deepen and broaden large language model assessment via structured evaluation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 5300–5318. Association for Computational Linguistics, 2024.
- [Fitting, 1985] Melvin Fitting. A kripke-kleene semantics for logic programs. *J. Log. Program.*, 2(4):295–312, 1985.
- [Greshake *et al.*, 2023] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023.
- [Gu *et al.*, 2024] Zhouhong Gu, Haoning Ye, Xingzhou Chen, Zeyang Zhou, Hongwei Feng, and Yanghua Xiao. StrucText-Eval: Evaluating large language model’s reasoning ability in structure-rich text. *arXiv preprint arXiv:2406.10621*, 2024.
- [Jain *et al.*, 2023] Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. Baseline defenses for adversarial attacks against aligned language models. *arXiv preprint arXiv:2309.00614*, 2023.
- [Khot *et al.*, 2022] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. *arXiv preprint arXiv:2210.02406*, 2022.
- [Liang *et al.*, 2023] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023.
- [Mukherjee *et al.*, 2013] Arjun Mukherjee, Vivek Venkataraman, Bing Liu, and Natalie S. Glance. What yelp fake review filter might be doing? In *Proceedings of the International AAAI Conference on Web and Social Media*. AAAI Press, 2013.
- [Perez and Ribeiro, 2022] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022.
- [Press *et al.*, 2023] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.
- [Qin *et al.*, 2023] Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, et al. Tool learning with foundation models. *arXiv preprint arXiv:2304.08354*, 2023.
- [Sun *et al.*, 2023] Weiwei Sun, Lingyong Yan, Xinyu Ma, Pengjie Ren, Dawei Yin, and Zhaochun Ren. Is chatgpt good at search? investigating large language models as re-ranking agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [Wang *et al.*, 2023a] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2023.
- [Wang *et al.*, 2023b] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.
- [Wang *et al.*, 2024] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jikai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 2024.
- [Wei *et al.*, 2022] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc., 2022.
- [Yao *et al.*, 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [Zhou *et al.*, 2023] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations*, 2023.

A Extended Related Works

B method detail

C Experiment Details

C.1 Data Construction

Restaurant Selection. We select restaurants from the Yelp Open Dataset using a multi-stage pipeline: (1) Filter by metropolitan area and target category (cafes); (2) Rank candidates by review volume to ensure sufficient evidence; (3) Apply LLM-based category verification to filter false positives. We retain 20 restaurants meeting quality thresholds.

Review Sampling. For each restaurant, we construct a document set of up to 20 reviews. To capture diverse user sentiments, we stratify sampling by star rating, targeting 4 reviews from each level (1–5 stars) where available. Within each stratum, we select reviews by descending text length to maximize information density. All reviewer identifiers are anonymized.

C.2 Evidence Types

We support four evidence types for evaluating user request conditions:

item_meta. Direct lookup of structured item attributes (e.g., `attributes.WiFi`, `attributes.NoiseLevel`). Missing values yield “unknown” (0); boolean attributes map directly to ± 1 . Categorical attributes use declarative matching rules: if `true` condition matches, return $+1$; if `false` condition matches, return -1 ; otherwise return 0.

item_meta_hours. Time-window containment verification. Given business hours (e.g., “7:00–18:00”) and required range (e.g., “10:00–15:00”), returns $+1$ if the required range falls entirely within business hours, -1 otherwise.

review_text. LLM-based review analysis. An LLM judges each review independently for the target aspect, returning $+1$ (positive mention), 0 (neutral/not mentioned), or -1 (negative mention). Judgments are cached for reproducibility. The final result uses *symmetric consensus voting* with a 75% threshold:

$$\text{result} = \begin{cases} +1 & \text{if } \frac{n_+}{n_+ + n_-} \geq 0.75 \\ -1 & \text{if } \frac{n_-}{n_+ + n_-} \geq 0.75 \\ 0 & \text{otherwise (no consensus)} \end{cases} \quad (3)$$

review_meta. Reviewer and review-level metadata conditions. Supports filtering (e.g., elite reviewers only, reviews from 2018+, minimum star rating) and weighted aggregation (by review count, usefulness votes, or social connections).

C.3 Request Structure

Each request is represented as a Boolean expression tree. Leaf nodes specify individual conditions with aspect names and evidence specifications. Internal nodes apply Boolean operators (AND/OR) with three-valued semantics:

AND	+1	0	-1	OR	+1	0	-1
+1	+1	0	-1	+1	+1	+1	+1
0	0	0	-1	0	+1	0	0
-1	-1	-1	-1	-1	+1	0	-1

C.4 Request Groups

G01 (R00–R04): Flat / Simple Metadata. Five requests using only direct item attributes with flat AND composition. Example: “Looking for a quiet cafe with free Wi-Fi and no TV.”

G02 (R05–R09): Mixed / Item Meta + Review Text. Five requests combining structured metadata filters with unstructured review text analysis. Example: “Budget-friendly indoor cafe where reviews mention good matcha.”

G03 (R10–R14): Flat / Computed Metadata. Five requests requiring hours interpretation. Example: “Quiet cafe with free Wi-Fi open Monday 10am–3pm.”

G04 (R15–R19): Flat / Social Metadata. Five requests using reviewer credibility signals. Example: “Places where experienced Yelp reviewers gave high ratings.”

C.5 Ground Truth and Evaluation

Label Generation. Ground truth labels are precomputed deterministically. For each (restaurant, request) pair: (1) Evaluate each condition against available evidence; (2) Aggregate results using the request’s Boolean structure; (3) Store the final label ($+1$, 0, or -1) with full evidence breakdown. LLM judgments for review text are cached to ensure reproducibility across runs. The resulting dataset contains 400 labeled pairs (20 restaurants $\times$ 20 requests).

Top-1 Selection. Each request is designed to have exactly one restaurant with a $+1$ (recommend) ground truth label. This enables unambiguous evaluation: for each request, there is a single correct answer among the 20 candidates.

Hits@k Metric. We evaluate recommendation accuracy using Hits@k:

$$\text{Hits@k} = \frac{1}{|R|} \sum_{r \in R} \mathbf{1}[\text{rank}(r) \leq k] \quad (4)$$

where R is the set of requests and $\text{rank}(r)$ is the position of the ground-truth positive item in the method’s ranking for request r . Hits@1 measures whether the method’s top recommendation exactly matches the gold item.

C.6 Baselines

We evaluate against 13 zero-shot prompting baselines organized into two categories.

Reasoning-Based Methods

Chain-of-Thought (CoT). Zero-shot prompting where the LLM analyzes all candidates in a single context window and selects the best match. Given a user request and the set of 20 candidate restaurants (each with metadata and reviews), the LLM is prompted to: (1) analyze each restaurant’s suitability for the user’s stated requirements; (2) select the single best-matching restaurant from the candidates. No few-shot examples are provided, testing the model’s inherent reasoning capabilities.

Self-Consistency (CoT-SC). Extends Chain-of-Thought by sampling multiple independent reasoning paths (typically 5) and selecting the final answer via majority vote. This reduces sensitivity to any single reasoning chain’s errors.

ReAct. Interleaves reasoning (Thought) with actions (examine reviews, check ratings, analyze requirements) in a multi-turn loop. The model alternates between thinking about the problem and taking actions to gather information, with a maximum of 5 steps before making the final selection.

Least-to-Most (L2M). Decomposes the user request into a sequence of simpler subproblems, solves each subproblem incrementally (with solutions to earlier subproblems available as context), then synthesizes the final recommendation from all partial solutions.

577 **Plan-and-Solve (PS).** Generates an explicit multi-step plan
578 before execution. The model first devises a strategy for eval-
579 uating candidates, then systematically follows the plan to an-
580 alyze each restaurant against the user’s requirements.

581 **Self-Ask.** Generates follow-up questions about the user re-
582 quest (e.g., “What specific attributes should I check for this
583 requirement?”), answers each question using the available ev-
584 idence, then uses the accumulated answers to make the final
585 selection.

586 **Decomposed Prompting.** Uses specialized sub-task han-
587 dlers orchestrated by a decomposer. Handlers include: `ex-`
588 `tract_requirements` (parse user constraints), `analyze_reviews`
589 (`assess_review_sentiment`), `count_evidence` (tally support-
590 ing/opposing mentions), and `make_decision` (synthesize final
591 ranking). Each handler focuses on a specific capability, im-
592 proving modularity.

593 **Ranking-Based Methods**

594 **Pointwise (Fine-Grained).** Scores each candidate indepen-
595 dently on a 4-point relevance scale: Highly Relevant (3), Rel-
596 evant (2), Somewhat Relevant (1), Not Relevant (0). Can-
597 didates are then ranked by their scores. Complexity: $O(n)$
598 LLM calls for n candidates.

599 **PaRaDe.** Few-shot demonstration-based ranking that pro-
600 vides explicit examples of how to compare and rank candi-
601 dates. The demonstrations guide the model toward consistent
602 ranking behavior.

603 **Pairwise (PRP).** Compares all pairs of candidates with the
604 prompt “Which is more relevant to the user’s request: A or
605 B?” Aggregates pairwise wins into a total score for each can-
606 didate, then ranks by win count. Complexity: $O(n^2)$ LLM
607 calls for n candidates.

608 **RankGPT.** Sliding-window listwise reranking that pro-
609 cesses candidates in overlapping windows. The model ranks
610 items within each window, and the process iterates to refine
611 the global ranking. Balances context length constraints with
612 ranking quality.

613 **Setwise.** Compares candidates in sets (typically 3–5 items)
614 rather than pairs. For each set, the model identifies the best
615 candidate; this process repeats with tournament-style elimi-
616 nation. Offers a middle ground between pairwise granularity
617 and listwise efficiency.

618 **Listwise.** Single-pass ranking where the LLM outputs a
619 complete permutation of all candidates in one response. The
620 model sees all items simultaneously and produces the full
621 ranking directly. Complexity: $O(1)$ LLM calls (single call),
622 but requires sufficient context length.